

TEST YOURSELF: KNOWLEDGE VERIFICATION



UNIVERSIDAD

Escuela Colombiana de Ingeniería Julio Garavito

Project: Lab 05 - Reverse Engineering (Questionnaire)

Subject: IT Security and Privacy

Professor: Daniel Esteban Vela López

Authors: Andersson David Sánchez Méndez & Cristian Santiago Pedraza Rodríguez

Date: 2026-02-24

Location: Bogotá, Colombia

Confidentiality: Internal / Academic Use Only



1. Conceptual: Disassembler vs. Debugger

1.1 Main Differences

The fundamental difference lies in **execution state**.

- ♦ A **Disassembler** (e.g., `objdump`, `Cutter` in static mode) translates machine code into assembly language without running the program. It provides a "static map" of the binary.
- ♦ A **Debugger** (e.g., `GDB`) executes the binary in a controlled environment, allowing the analyst to pause execution, inspect memory, and modify registers in real-time.

1.2 Use Cases

- ♦ **Use a Disassembler when:** You need to understand the global structure of the program, identify hardcoded strings, find function entry points, or analyze a binary that might be malicious and shouldn't be executed yet.

- ◆ **Use a Debugger when:** You need to see how the program behaves with specific inputs, identify the value of variables at a specific moment (like the password comparison in this lab), or trace the exact path taken during a conditional jump (`JE` / `JNE`).



2. Practice: Analysis Methodology for Unknown Binaries

Given an unknown binary, our methodology follows a progressive "Outside-In" approach:

1. **Identification (`file`):** Determine the architecture (x86, ARM), format (ELF, PE), and whether the binary is "stripped" or contains symbols.
2. **Information Gathering (`strings`):** Search for plaintext passwords, IP addresses, developer comments, or UI messages that reveal the program's intent.
3. **Static Mapping (`objdump` / `Cutter`):** Disassemble the code to locate the `main` function and identify the primary logical flow and library calls (`printf` , `strcmp` , etc.).
4. **Dynamic Analysis (`GDB`):** Set breakpoints at identified "Decision Points" (conditional jumps). Run the program and inspect registers (like `RAX` or `RSP`) to see the data being compared.
5. **Behavioral Verification:** Correlate the visual graph (flowchart) with real-time memory values to confirm the bypass or exploit vector.



3. Ethics: Legal and Moral Boundaries

3.1 Legal Circumstances

Reverse engineering commercial software is generally permitted under specific circumstances (varying by jurisdiction, such as DMCA in the US or EU Directives):

- ◆ **Interoperability:** Creating software that works with the original product.
- ◆ **Security Research:** Identifying vulnerabilities to report them and improve the ecosystem.
- ◆ **Education:** Learning how specific algorithms or compiler optimizations function.

3.2 Required Precautions

To remain within ethical and safe boundaries, an analyst should:

- ◆ **Isolated Environments:** Always perform analysis in a Virtual Machine (VM) or sandbox to prevent accidental system damage or malware spread.
- ◆ **Avoid Distribution:** Do not distribute cracked versions or proprietary code extracted during the process.
- ◆ **Responsible Disclosure:** If a vulnerability is found, report it to the vendor first (Bug Bounty programs) before making it public.
- ◆ **Non-Disclosure Agreements (NDAs):** Respect any legal contracts signed that explicitly prohibit the deconstruction of the software.



4. Conclusions of the Exercise

The transition from conceptual theory to practical execution highlights that security cannot rely on the secrecy of the binary. Every decision gate—no matter how complex—can be identified and analyzed through the proper application of the tools used in this laboratory.

